

Product Requirements Document

Hello World API Enhancement + Get Persons Endpoint

Prepared as a practical API training / real-job feature specification.

1. Product Name

Hello World API

2. Purpose

The purpose of this API is to provide a simple endpoint that confirms the application is running correctly, while also serving as a training foundation for professional API development patterns.

- API routing
- Controllers
- DTOs
- Service layer design
- `ServiceResult<T>` pattern
- Validation and error handling
- Swagger testing

3. Current State

The API currently has a basic Hello World endpoint.

Existing Endpoint

`GET /api/hello`

Existing Response Example

`"Hello World"`

4. Feature 1: Personalized Greeting Endpoint

Add an enhanced Hello World endpoint that accepts a user name and returns a personalized greeting response.

Business Requirement

As an API user, I want to send a name to the Hello World API so that I can receive a personalized greeting message.

User Story

As an API user, I want to pass my name into the Hello World API, so that I can receive a custom greeting message.

Endpoint

```
GET /api/hello/{name}
```

Example:

```
GET /api/hello/Franky
```

Successful Response

```
{  
  "message": "Hello, Franky!"  
}
```

Response DTO

```
public class HelloResponseDto  
{  
    public string Message { get; set; } = string.Empty;  
}
```

Validation Rules

Rule	Expected Result
Name is empty	400 Bad Request
Name is whitespace	400 Bad Request
Name is longer than 50 characters	400 Bad Request
Name is valid	200 OK

Service Layer Requirement

```
Task<ServiceResult<HelloResponseDto>> GetGreetingAsync(string name);
```

The service should validate the name and return `ServiceResult<HelloResponseDto>`. The controller should not contain business validation logic.

Controller Requirement

```
[HttpGet("{name}")]  
public async Task<IActionResult> GetGreeting(string name)  
{  
    var result = await _helloService.GetGreetingAsync(name);  
    return result.ToActionResult();  
}
```

Acceptance Criteria

- Valid name returns 200 OK with { "message": "Hello, Franky!" }.
- Blank or whitespace name returns 400 Bad Request with "Name is required."
- Name longer than 50 characters returns 400 Bad Request with "Name cannot be longer than 50 characters."

5. Feature 2: Get Persons Endpoint

Add a new endpoint to the existing Hello controller that returns a paged list of persons. The endpoint allows optional filtering by name, sorting, and pagination.

Purpose

- Practice query parameters
- Practice filtering by name
- Practice sorting and paging
- Practice service-layer validation
- Keep the controller thin with `ServiceResult<T>` and `ToActionResult()`

Endpoint

GET /api/hello/persons

Query Parameters

Parameter	Type	Required	Description
name	string	No	Optional name filter
sortBy	string	No	Field used for sorting
sortDir	string	No	Sort direction
pageNumber	int	Yes	Page number to return
pageSize	int	Yes	Number of records per page

Service Method Requirement

```
Task<ServiceResult<PagedResponse<PersonDto>>> GetPersonsAsync(
    string name,
    string sortBy,
    string sortDir,
    int pageNumber,
    int pageSize);
```

If the project does not already have `PagedResponse<T>`, the response can temporarily return `ServiceResult<List<PersonDto>>`. The preferred real-world version is `PagedResponse<PersonDto>`.

DTO Requirements

```
public class PersonDto
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public int Age { get; set; }
}

public class PagedResponse<T>
{
    public List<T> Data { get; set; } = new();
    public int PageNumber { get; set; }
    public int PageSize { get; set; }
}
```

```
        public int TotalCount { get; set; }
        public int TotalPages { get; set; }
    }
```

Validation Rules

sortBy Validation

Allowed values: **id**, **name**, **age**. If sortBy is empty, null, or whitespace, default to **name**. If sortBy is not allowed, return 400 Bad Request.

sortDir Validation

Allowed values: **asc**, **desc**. If sortDir is empty, null, or whitespace, default to **asc**. If sortDir is not allowed, return 400 Bad Request.

pageNumber Validation

pageNumber must be greater than 0. Invalid values return 400 Bad Request with: **Page number must be greater than 0.**

pageSize Validation

pageSize must be greater than 0 and less than 10. Valid values are 1 through 9. Invalid values include 0, 10, and 25.

```
    Page size must be greater than 0 and less than 10.
```

6. Get Persons Request / Response Examples

Valid Request

```
GET /api/hello/persons?name=frank&sortBy=name&sortDir=asc&pageNumber=1&pageSize=5
```

Successful Response Example

```
{
  "data": [
    {
      "id": 1,
      "name": "Franky",
      "age": 46
    }
  ],
  "pageNumber": 1,
  "pageSize": 5,
  "totalCount": 1,
  "totalPages": 1
}
```

Invalid sortBy

```
GET /api/hello/persons?sortBy=email&sortDir=asc&pageNumber=1&pageSize=5
```

```
{
  "errors": [
    {
      "type": "Validation",
      "message": "sortBy must be one of the following values: id, name, age."
    }
  ]
}
```

Invalid sortDir

```
GET /api/hello/persons?sortBy=name&sortDir=up&pageNumber=1&pageSize=5
```

```
{
  "errors": [
    {
      "type": "Validation",
      "message": "sortDir must be either asc or desc."
    }
  ]
}
```

Invalid pageNumber

```
GET /api/hello/persons?sortBy=name&sortDir=asc&pageNumber=0&pageSize=5
```

```
{
  "errors": [
    {
      "type": "Validation",
      "message": "Page number must be greater than 0."
    }
  ]
}
```

Invalid pageSize

```
GET /api/hello/persons?sortBy=name&sortDir=asc&pageNumber=1&pageSize=10
```

```
{
  "errors": [
    {
      "type": "Validation",
      "message": "Page size must be greater than 0 and less than 10."
    }
  ]
}
```

7. Multi-Error Validation Requirement

The service should collect validation errors before returning. It should return all validation errors together instead of stopping after the first error.

```
GET /api/hello/persons?sortBy=email&sortDir=up&pageNumber=0&pageSize=10

{
  "errors": [
    {
      "type": "Validation",
      "message": "sortBy must be one of the following values: id, name, age."
    },
    {
      "type": "Validation",
      "message": "sortDir must be either asc or desc."
    },
    {
      "type": "Validation",
      "message": "Page number must be greater than 0."
    },
    {
      "type": "Validation",
      "message": "Page size must be greater than 0 and less than 10."
    }
  ]
}
```

8. Controller Requirement

The controller should stay thin. It should pass parameters to the service and return `result.ToActionResult()`.

```
[HttpGet("persons")]
public async Task<IActionResult> GetPersons(
    string name,
    string sortBy,
    string sortDir,
    int pageNumber,
    int pageSize)
{
    var result = await _helloService.GetPersonsAsync(
        name,
        sortBy,
        sortDir,
        pageNumber,
        pageSize);

    return result.ToActionResult();
}
```

9. Get Persons Acceptance Criteria

- Valid query parameters return 200 OK and a paged list of persons.
- Unsupported sortBy returns 400 Bad Request with allowed sortBy values.
- Unsupported sortDir returns 400 Bad Request and says sortDir must be asc or desc.
- pageNumber <= 0 returns 400 Bad Request and says page number must be greater than 0.
- pageSize <= 0 or pageSize >= 10 returns 400 Bad Request and says page size must be greater than 0 and less than 10.
- Multiple invalid values return all validation errors together.

10. Technical Tasks

- Create PersonDto.
- Add GetPersonsAsync to IHelloService.
- Implement GetPersonsAsync in HelloService.
- Add allowed sortBy values: id, name, age.
- Add allowed sortDir values: asc, desc.
- Default empty sortBy to name.
- Default empty sortDir to asc.
- Validate pageNumber > 0.
- Validate pageSize > 0 && pageSize < 10.
- Collect all validation errors before returning.
- Return ServiceResult>.
- Add GET /api/hello/persons to HelloController.
- Return result.ToActionResult() from the controller.
- Test valid request in Swagger.
- Test invalid sortBy in Swagger.
- Test invalid sortDir in Swagger.
- Test invalid pageNumber in Swagger.
- Test invalid pageSize in Swagger.
- Test multiple validation errors in Swagger.

11. Definition of Done

- GET /api/hello/persons exists.
- The controller calls GetPersonsAsync.
- The service validates sortBy, sortDir, pageNumber, and pageSize.

- Invalid values return 400 Bad Request.
- Multiple validation errors are returned together.
- Valid requests return 200 OK.
- The controller remains thin.
- The endpoint has been tested in Swagger.

12. Suggested Folder Structure

```
Controllers
  HelloController.cs

DTOs
  HelloResponseDto.cs
  PersonDto.cs
  PagedResponse.cs

Services
  IHelloService.cs
  HelloService.cs

Common
  ServiceResult.cs
  ServiceError.cs
  ServiceErrorType.cs
  ServiceResultExtensions.cs
```